Common Pitfalls: Data Leakage and Class Imbalance

When excellent-looking numbers cannot be trusted

Model Evaluation · Foundations of Machine Learning

Data Leakage: Information From the Wrong Time

Definition — Information unavailable at prediction time influences training or feature construction.

Result — Evaluation looks excellent; deployment performance collapses.

Diagnostic question — Would this exact information exist, in this exact form, at the moment the model must predict?

1. Full data (train + test together)
2. Transform fit on everything, test included
3. Train and evaluate as usual
4. Inflated score — the test set has already leaked in

Split Before You Fit Preprocessing

Wrong

Fit scaling, imputation, or feature selection on the full dataset, then split into train/test.

Right

Split first; fit every transform only on the training data; apply the fitted transform unchanged to validation/test.

Best guardrail

Use a scikit-learn Pipeline so cross-validation automatically refits preprocessing inside each training fold.

A Pipeline Enforces the Boundary

Fit

Each cross-validation fold learns its own scaling statistics from its own training rows.

Transform

Held-out rows for that fold are transformed, never used to compute the statistics.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

model = make_pipeline(
    StandardScaler(), LogisticRegression(),
)
scores = cross_val_score(model, X, y, cv=5)
assert np.isfinite(scores).all()
```

How Much Does Leakage Actually Inflate a Score?

Setup: 200 examples, 5,000 pure-noise features, labels assigned independently at random — the true relationship between X and y is **zero**. Any signal a model finds is illusory.

```
# LEAKY: select top-20 features using ALL data
selector = SelectKBest(f_classif, k=20)
selector.fit(X, y)                # sees y for every row
X_sel = selector.transform(X)
leaky = cross_val_score(
    LogisticRegression(), X_sel, y, cv=5,
)  # mean ≈ 0.81

# HONEST: selection refit inside each fold
pipe = make_pipeline(
    SelectKBest(f_classif, k=20), LogisticRegression(),
)
honest = cross_val_score(pipe, X, y, cv=5)
# mean ≈ 0.43 — chance is 0.50
```

Leaky pipeline

≈ 0.81 accuracy

Feature selection saw the labels for every row, including the "held-out" fold, before cross-validation began — it hand-picked 20 noise columns that happen to correlate with y by chance.

Honest pipeline

≈ 0.43 accuracy

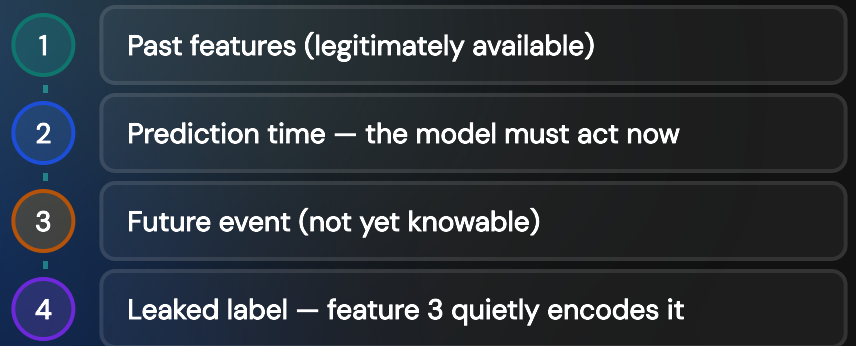
Selection is refit inside each fold using only that fold's training rows; performance correctly lands near the 0.50 chance rate for a label with no real signal.

Target and Temporal Leakage

Target proxy — A feature indirectly encodes the label, e.g. "sent to collections" as a feature for predicting loan default.

Future information — A feature's time window extends past the moment of prediction, e.g. using a patient's full hospital stay to predict admission.

Symptom — Suspiciously high performance, or one feature with implausibly enormous importance, that cannot survive real timing constraints.



Class Imbalance Distorts Training and Evaluation

Evaluation — Accuracy can ignore near-total failure on a rare class (the accuracy paradox, revisited).

Training — The majority class dominates average empirical risk, so the optimizer barely notices minority-class errors.

Consequence — The fitted model learns that majority-favoring errors are cheap and minority errors are nearly free.

$$\widehat{R}(\theta) = \frac{1}{n} \sum_{i} \ell(f_\theta(x_i), y_i)$$

With 990 majority and 10 minority examples, minority mistakes contribute only 10/1000 = 1% of the total sum — the optimizer has almost no incentive to fix them.

Mitigation Has Tradeoffs

Class weighting

Multiply minority-example loss contributions (e.g. class_weight="balanced") so the optimizer can no longer ignore them.

Resampling

Over-sample the minority class or under-sample the majority class before training.

Threshold tuning

Move the operating point off the default 0.5 to favor recall or precision, using the ROC/PR tradeoff from the metrics deck.

Metrics

Always report per-class precision, recall, and F1 — never accuracy alone — for an imbalanced problem.

Before You Ship

Preprocessing

Was every learned transform fit only on training data, inside a Pipeline?

Availability

Do all features exist, unaltered, at the real moment of prediction?

Metric

Does the reported metric expose minority-class performance, not just an aggregate?

Test discipline

Was the test set touched exactly once, after every modeling decision was already fixed?

Evaluation Can Fail Before the Metric

Leakage

Breaks information boundaries; can inflate scores by tens of points on pure noise.

Imbalance

Hides minority failure in aggregate metrics and starves it of training signal.

Pipelines + per-class metrics

Prevent the two most common evaluation failures structurally.

Next: ensemble methods combine trees for stronger predictions.